

GA7-220501096-AA3-EV01

Codificación de módulos del software Stand

alone, web y móvil de acuerdo al proyecto a desarrollar

Análisis y Desarrollo de Software (ADSO)

Ficha: 3134536

Instructora: Astrid Maritza Calvache

Diana Carolina Cruz Marin

Marzo de 2026

Introducción

En el desarrollo de software se requiere la implementación de soluciones tecnológicas que se adapten a diferentes entornos como aplicaciones web, móviles y sistemas independientes. En este trabajo se llevó a cabo la codificación de módulos de software teniendo en cuenta los requerimientos del proyecto propuesto, aplicando buenas prácticas de programación.

Durante el proceso se estructuró el proyecto de forma organizada, facilitando la conexión de la base de datos y la creación de funcionalidades que respondan a las necesidades planteadas. Además, se realizó la configuración del entorno para garantizar el correcto funcionamiento del sistema en diferentes equipos.

Objetivos

Objetivo general

Desarrollar la codificación de módulos de software para aplicaciones stand alone, web y móvil, de acuerdo con los requerimientos.

Objetivos específicos

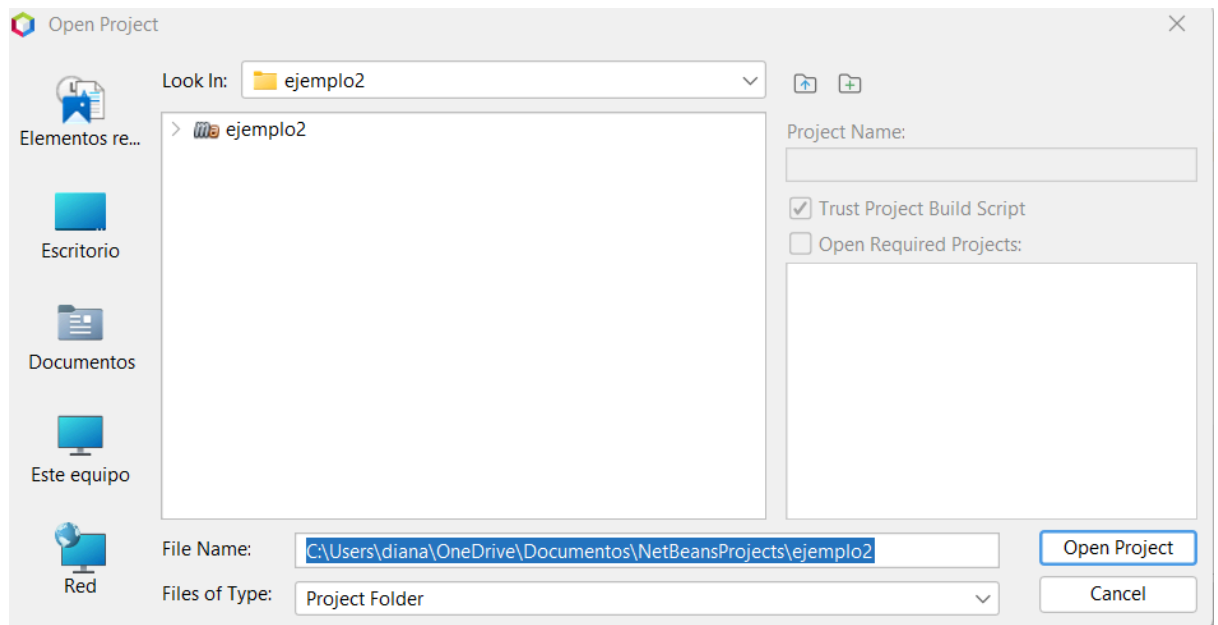
- Implementar la estructura del proyecto utilizando herramientas y frameworks adecuados.
- Desarrollar la conexión con la base de datos para el almacenamiento y gestión de la información.
- Codificar los módulos necesarios para el funcionamiento del sistema.
- Verificar el correcto funcionamiento del software mediante pruebas de ejecución.
- Documentar el proceso de desarrollo para facilitar su comprensión y uso.

Paso a paso

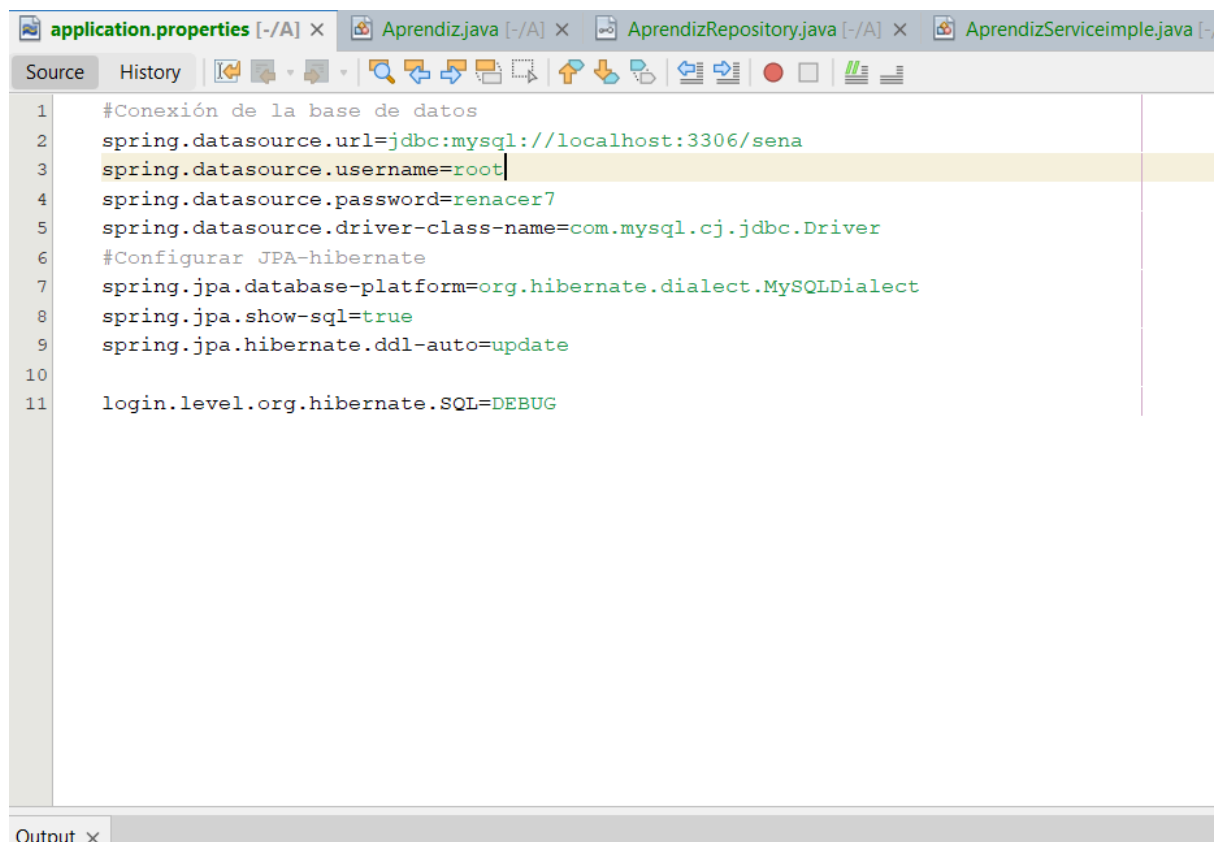
1. Ir a spring y configurar el proyecto, descargarlo y descomprimirlo, pero como tengo la versión 22 no tengo que descargarlo porque ya lo tengo.



2. Abrimos Netbeans y lo descomprimos.



3. Crear base de datos
4. En other resources buscamos el que diga application.properties y hacemos la conexión con la base de datos y configurar JPA- hibernate.



5. Crear paquete: model, repository, service y controller.

```

>  ServletInitializer.java [-/A]
>  com.Proyecto2.ejemplo2.Controller
>  com.Proyecto2.ejemplo2.Model
>  com.Proyecto2.ejemplo2.Repository
>  com.Proyecto2.ejemplo2.Service

```

6. Crear una java class llamada aprendiz y colocarle sus respectivas líneas de código.

```

- import jakarta.persistence.Column;
  import jakarta.persistence.Entity;
  import jakarta.persistence.Id;
  import lombok.Data;

- /**
  *
  * @author diana
  */
  @Entity
  @Data
  public class Aprendiz {
      //Crear elementos de la tabla Aprendiz
      @Id
      @Column
      private long iduser;
      @Column
      private String nomuser;
      @Column
      private String apellido;
      @Column
      private String email;
  }

```

7. En el paquete repository creamos agregamos una interfaz llamada aprendizRepository y creamos un código para establecer comunicación entre el paquete del modelo y el repositorio jpa.

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Interface.java to edit this template
4  */
5  package com.Proyecto2.ejemplo2.Repository;
6
7  import com.Proyecto2.ejemplo2.Model.Aprendiz;
8  import org.springframework.data.jpa.repository.JpaRepository;
9
10 /**
11  *
12  * @author diana
13  */
14 public interface AprendizRepository extends JpaRepository<Aprendiz, Long>{
15
16 }
17
```

8. En el paquete services creamos una interfaz llamada aprendizservices y dentro de él creamos los métodos: crear, borrar, modificar y leer.

```
package com.Proyecto2.ejemplo2.Service;

import com.Proyecto2.ejemplo2.Model.Aprendiz;

/**
 *
 * @author diana
 * Permite hacer la creación de los metodos:crear,borrar,modificar,leer
 */
public interface AprendizServices {
    Aprendiz newAprendiz (Aprendiz newAprendiz);
    Iterable<Aprendiz> getAll();//Busqueda
    Aprendiz modifyAprendiz (Aprendiz aprendiz);//Modificar
    Boolean deleteAprendiz (Long iduser);//Recibe falso y verdadero para saber si se elimino
}
```

9. En services creamos un java class llamado aprendiz serviceimple para hacer la implementación de los métodos creados en aprendizservice y un código para agregar un aprendiz a la base de datos y que se agregue al registro.

```

package com.Proyecto2.ejemplo2.Service;

import com.Proyecto2.ejemplo2.Model.Aprendiz;
import com.Proyecto2.ejemplo2.Repository.AprendizRepository;
import java.util.Optional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class AprendizServiceimple implements AprendizServices {
    @Autowired
    private AprendizRepository aprendizrepository;
    //Agregar un nuevo aprendiz
    @Override
    public Aprendiz newAprendiz(Aprendiz newAprendiz) {
        return aprendizrepository.save(newAprendiz);
    }
    //Encuentra todos los registros de la base de datos para que los muestre todos
    @Override
    public Iterable<Aprendiz> getAll() {
        return this.aprendizrepository.findAll();
    }

    public Iterable<Aprendiz> getAll(); {
        return this.aprendizrepository.findAll();
    }

    @Override
    //Modifica o actualiza un aprendiz
    public Aprendiz modifyAprendiz(Aprendiz aprendiz) {
        Optional<Aprendiz> aprendizEncontrado=this.aprendizrepository.findById(aprendiz.getIduser());
        if (aprendizEncontrado.get() !=null) {
            aprendizEncontrado.get().setNomuser(aprendiz.getNomuser());
            aprendizEncontrado.get().setApellido(aprendiz.getApellido());
            aprendizEncontrado.get().setEmail(aprendiz.getEmail());
            return this.newAprendiz(aprendizEncontrado.get());
        }
        return null;
    }

    @Override
    //Eliminar por ID
    public Boolean deleteAprendiz(Long iduser) {
        this.aprendizrepository.deleteById(iduser);
        return true;
    }
}

```

10. En controller crear java class aprendizcontroller e importar los siguientes paquetes.

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.Proyecto2.ejemplo2.Controller;

import com.Proyecto2.ejemplo2.Model.Aprendiz;
import com.Proyecto2.ejemplo2.Service.AprendizServices;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/aprendiz")
public class Aprendizcontroller {
    @Autowired
    private AprendizServices aprendizServices;

    //Para crear un nuevo aprendiz

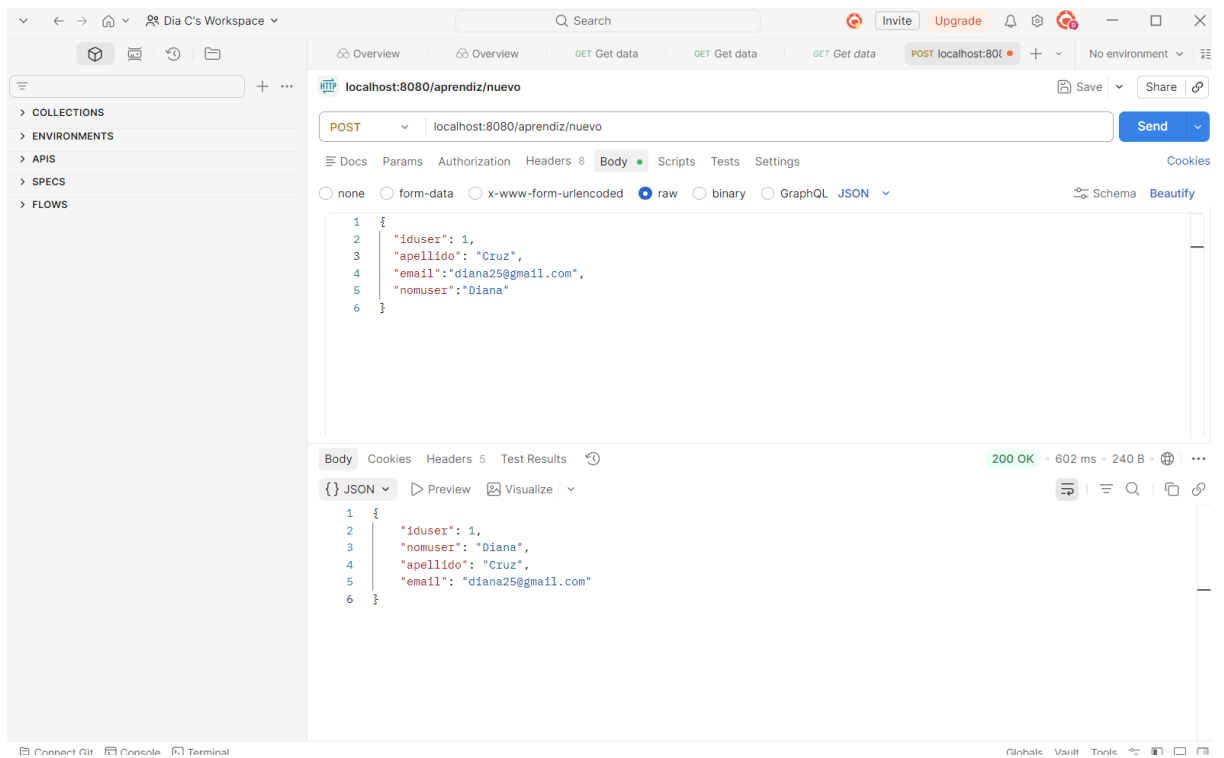
```

```

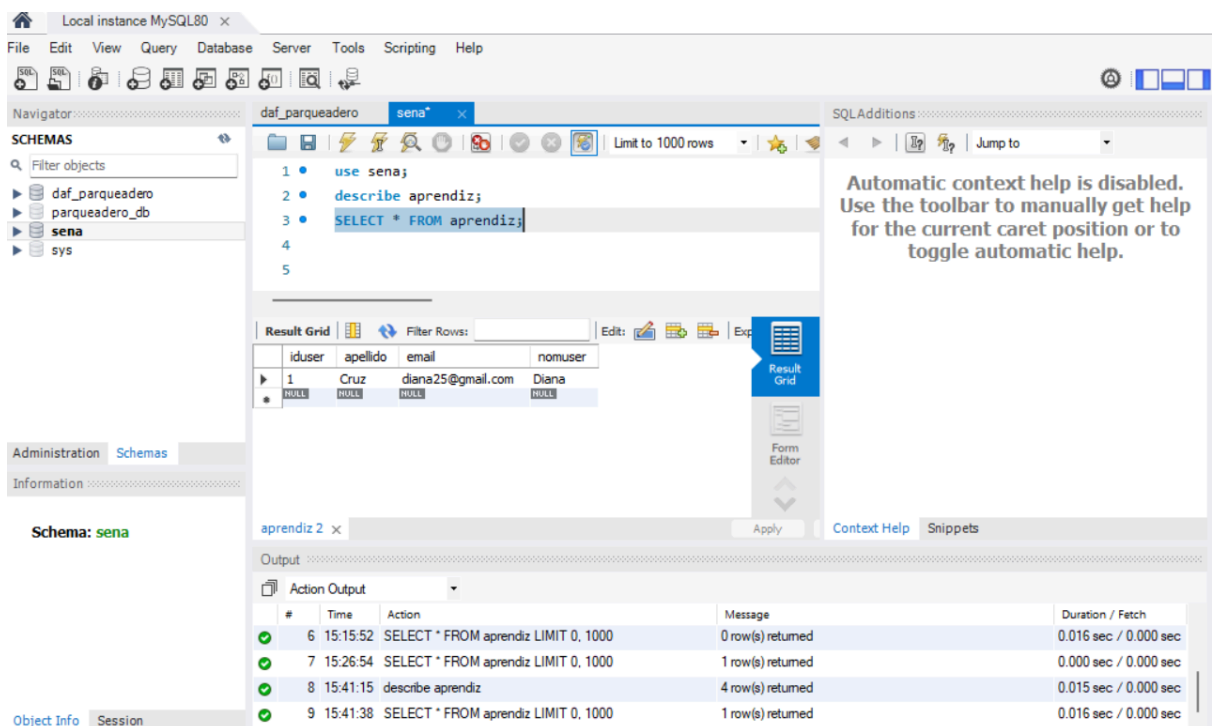
Source History
23 //Para crear un nuevo aprendiz
24 @PostMapping("/nuevo")
25 public Aprendiz newAprendiz(@RequestBody Aprendiz aprendiz){
26     return this.aprendizServices.newAprendiz(aprendiz);
27
28 //Para mostrar la información contenida en la base de datos
29 @GetMapping("/mostar")
30 public Iterable<Aprendiz> getAll(){
31     return aprendizServices.getAll();
32
33 //Pra modificar
34 @PostMapping("/modificar")
35 public Aprendiz updateAprendiz(@RequestBody Aprendiz aprendiz){
36     return this.aprendizServices.modifyAprendiz(aprendiz);
37
38 //Borra aprendiz por id
39 @PostMapping (value="/{id}")
40 public Boolean deleteApreniz (@PathVariable(value="id")Long id){
41     return this.aprendizServices.deleteAprendiz(id);
42
43
44 }
45

```

11. Buscar project files/pom.xml y eliminamos la dependencia tomcat
12. le damos run file.
13. Descargamos postman y nos conectamos con nuestra cuenta de gmail.
14. Agregamos datos en postman.



15. Ahora aparecen en la base de datos



Conclusión

En conclusión, el desarrollo de esta actividad permitió aplicar los conocimientos adquiridos en la codificación de módulos de software, así como el uso de herramientas. Se logró implementar un sistema funcional que cumple con los requerimientos establecidos, evidenciando la importancia de una correcta configuración del entorno y la estructura del proyecto.

Además, se comprende la relevancia de realizar pruebas y ajustes durante el proceso, ya que esto garantiza que el software pueda ejecutarse correctamente en diferentes equipos. Finalmente, esta experiencia contribuyó al fortalecimiento de habilidades en el desarrollo de aplicaciones y en la resolución de problemas técnicos.